



14

并查集

Business templates allow you to enjoy flying. Business templates allow you to enjoy flying



问题的提出



- 并查集是一种树型的数据结构
- 用于集合的查询，合并(连通块的处理)
- 问题提出：
 - 给出 n 个点， m 条边，组成一片森林。
 - 多次问 x 和 y 两个点是否处于同一棵树上，输出YES，如果不在同一棵树上，输出NO，并把 x 和 y 所在的两棵树合并成一棵树。

① 集合的建立

② 集合元素的查询

③ 集合的合并



01

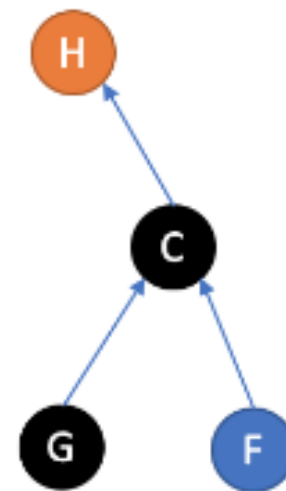
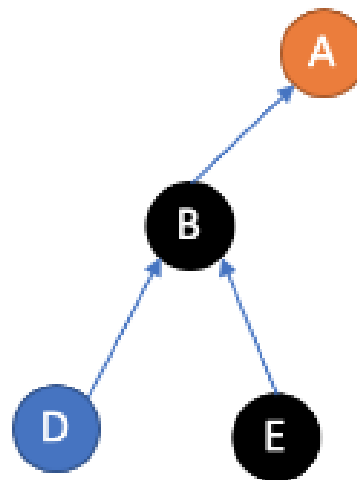
◆ 并查集的实现 ◆

集合建立: $fa[] \text{-----} fa[i]=i$

元素所在集合的查询:

$find(x)$ 不断的向上查询 x 的父亲，直到找到祖先。

```
int find(int x){  
    if(fa[x]==x) return x;  
    return find(fa[x]);  
}
```



◆ 并查集的实现 ◆

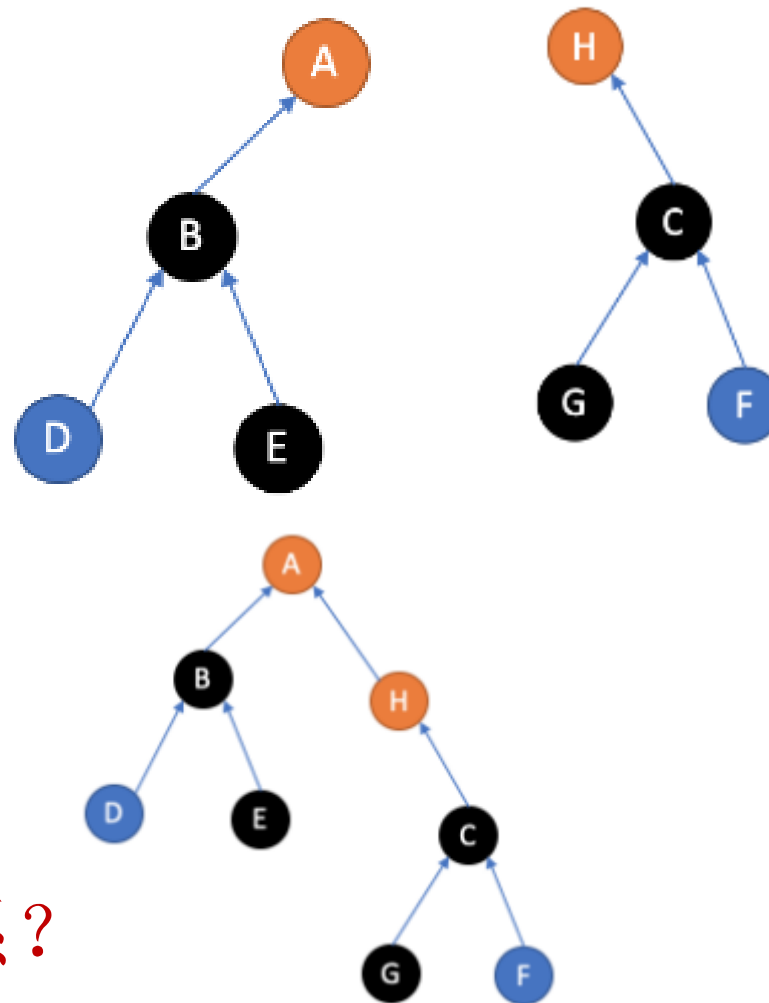
集合建立: $fa[] \text{-----} fa[i]=i$

集合的合并:

如果两人的祖先不同，则让祖先建立父子关系。

```
void union(int x,int y){  
    int fx=find(x);  
    int fy=find(y);  
    if(fx!=fy)fa[fx]=fy;  
}
```

在查询祖先时，时间复杂度什么有关系？



◆ 并查集的优化 ◆

- 启发式合并：记录每个集合的节点个数，然后把深度小的集合合并到节点多的集合上。最坏的情况，是每次合并的时候，两个集合的深度 h 都一样。新的集合深度 $h+1$ 。
- 路径压缩：就是在查询一个节点所在的集合时，当查询到集合的顶级元素后，在回溯的时把所有的节点区别指向顶级元素，以便提高下次查询效率。

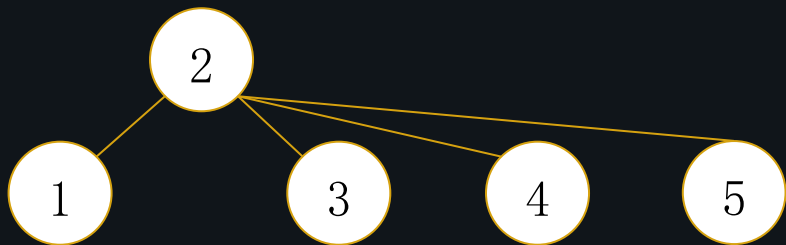




路径压缩实现



路径压缩实际上是在找完根结点之后，在递归回来的时候顺便把路径上元素的父亲指针都指向根结点。这就是说，我们每一次找祖先（**find**），都可能产生一次路径压缩



元素所在集合的查询:

```
int find(int x)
```

```
{
```

```
    if(fa[x]==x) return x;
```

```
    fa[x]=find(fa[x]);
```

```
    return fa[x];
```

```
}
```



犯罪团伙



- 警察抓到了 n 个罪犯，警察根据经验知道他们属于不同的犯罪团伙，却不能判断有多少个团伙，但通过警察的审讯，知道其中的一些罪犯之间相互认识，已知同一犯罪团伙的成员之间直接或间接认识。有可能一个犯罪团伙只有一个人。请你根据已知罪犯之间的关系，确定犯罪团伙的数量。已知罪犯的编号从1至 n 。
- 输入：
- 第一行： n (≤ 1000 , 罪犯数量)，
- 第二行： m (≤ 5000 , 关系数量)
- 以下若干行：每行两个数： i 和 j ，中间一个空格隔开，表示罪犯 i 和罪犯 j 相互认识。
- 输出：
- 一个整数，犯罪团伙的数量。



P1551 亲戚



- 题目描述

规定：x和y是亲戚，y和z是亲戚，那么x和z也是亲戚。如果x,y是亲戚，那么x的亲戚都是y的亲戚，y的亲戚也都是x的亲戚。

- 输入格式：

第一行：三个整数n,m,p，（ $n \leq 5000, m \leq 5000, p \leq 5000$ ），分别表示有n个人，m个亲戚关系，询问p对亲戚关系。

以下m行：每行两个数 M_i, M_j ， $1 \leq M_i, M_j \leq N$ ，表示 M_i 和 M_j 具有亲戚关系。

接下来p行：每行两个数 P_i, P_j ，询问 P_i 和 P_j 是否具有亲戚关系。

- 输出格式：

P行，每行一个' Yes'或' No'。表示第i个询问的答案为“具有”或“不具有”亲戚关系。



并查集扩展



并查集一般作用：

- 1、维护节点之间的连通性及求联通快
- 2、动态维护许多具有传递性的关系

并查集实际上是有若干颗树构成的森林，我们可以在树中的每条边上记录一个权值，即维护一个数组 d ，用 $d[x]$ 保存节点 x 到父节点 $fa[x]$ 之间的边权。在每次路径压缩后，每个访问过的节点就会直接指向树根，如果我们同时更新这些节点的 d 值，就可以利用路径压缩过程来统计每个节点到树根之间路径上的一些信息。这就是所谓的“边带权”的并查集。



带权值并查集



P1196 银河英雄传说

有一个划分成 N 列的星际战场，各列依次编号为 $1, 2, \dots, N$ 。有 N 艘战舰，也依次编号为 $1, 2, \dots, N$ ，其中第 i 号战舰处于第 i 列。有 M 条指令：

1 M_{ij} ：表示让第 i 号战舰所在列的全部战舰保持原有顺序，接在第 j 号战舰所在列的尾部，并且保证第 i 号战舰与第 j 号战舰不在同一列。

2 C_{ij} ：表示询问第 i 号战舰与第 j 号战舰是否处于同一列中，如果在同一列中，他们之间间隔了多少艘战舰？

$N \leq 30000, m \leq 5 \times 10^5$



问题分析

利用并查集来处理：

一开始把 N 个战舰看成 N 个独立的集合。在没有路径压缩的情况下， $fa[x]$ 就表示排在第 x 号战舰前面的那个战舰的编号。一个集合的代表(集合的根)就是位于最前面的战舰。另外，让树上每条边带权值为1，这样树上两点之间的距离减1就是二者之间间隔的战舰数量。

在考虑路径压缩的情况下，我们额外建立一个数组 d ， $d[x]$ 记录战舰 x 与 $fa[x]$ 之间的边的权值。在路径压缩把 x 直接指向树根的同时，我们把 $d[x]$ 更新为从 x 到树根的路径上的所有边权和。



NOI2002



查询：对d数组的维护

```
int find(int x){  
    if(x==fa[x]) return x;  
    int root=find(fa[x]);  
    d[x]+=d[fa[x]];  
    return fa[x]=root;  
}
```

对于C指令，分别进行find(x)和find(y)完成查询和路径压缩，若二者的返回值相同，这说明x和y处于同一列中。所以d[x]保存了位于x之前的战舰数量，d[y]保存了位于y之前的战舰数量。二者只差减1，就是x和y之间间隔的战舰数量

合并：对d数组和size数组的维护

```
void merge(int x,int y){  
    x=find(x);y=find(y);  
    fa[x]=y;d[x]=size[y];  
    size[y]+=size[x];  
}
```

对于M指令，把x的树根作为y的树根的子节点，连接新边的权值应该为合并之间集合y的大小(依据题意，集合y中全部战舰都排在集合x之前)。因此，我们还需要一个size数组来记录每个树根上集合的大小。



带权值并查集



P1682 关押罪犯

S城现有两座监狱，一共关押着 N 名罪犯，编号分别为 $1-N$ 。很多罪犯之间积怨已久，我们用“怨气值”（一个正整数值）来表示某两名罪犯之间的仇恨程度。如果两名怨气值为 c 的罪犯被关押在同一监狱，他们俩之间会发生摩擦，并造成影响力为 c 的冲突事件。

假设只要处于同一监狱内的某两个罪犯间有仇恨，那么他们一定会在每年的某个时候发生摩擦。现在有两个监狱。那么，应如何分配罪犯，才能使市长看到的那个冲突事件的影响力最小？这个最小值是多少？



关押罪犯



输入 #1

```
4 6
1 4 2534
2 3 3512
1 2 28351
1 3 6618
2 4 1805
3 4 12884
```

输出 #1

```
3512
```

问题分析

利用贪心思想，优先分开怨气大的两个罪犯
,x,y，并建立敌对关系，

`enemy[x]=y;enemy[y]=x;`

只有两个监狱，敌人的敌人一定在同一监狱。

这时可建立朋友关系（用并查集），

`fa[x]=find(enemy[y]);`

第一次出现不能建立敌人关系时，这两个罪犯就不能分开了。可输出答案。



关押罪犯



输入 #1

```
4 6
1 4 2534
2 3 3512
1 2 28351
1 3 6618
2 4 1805
3 4 12884
```

输出 #1

```
3512
```

```
for(int i=1;i<=m;i++)scanf("%d%d%d",&edge[i].u,
sort(edge+1, edge+m+1, cmp);
int u, v, fu, fv;
for(int i=1;i<=m;i++)
{
    u=edge[i].u;v=edge[i].v;
    fu=find(u);fv=find(v);
    if(fu==fv)
    {
        printf("%d\n", edge[i].w);
        return 0;
    }
    if(enemy[u]==0) enemy[u]=v;
    else fa[fv]=find(enemy[u]);
    if(enemy[v]==0) enemy[v]=u;
    else fa[fu]=find(enemy[v]);
}
printf("0\n");
return 0;
```




种类并查集



P2024 食物链noi2011

动物王国中有三类动物 A,B,C，这三类动物的食物链构成了有趣的环形。A 吃 B，B 吃 C，C 吃 A。现有 N 个动物，以 1 — N 编号。每个动物都是 A,B,C 中的一种，但我们并不知道它到底是哪一种。用两种说法对这 N 个动物的食物链关系进行描述：第一种说法是“1 X Y”，表示 X 和 Y 是同类。

第二种说法是“2 X Y”，表示 X 吃 Y。

此人对 N 个动物，用上述两种说法，一句接一句地说出 K 句话，这 K 句话有的是真的，有的是假的。当一句话满足下列三条之一时，这句话就是假话，否则就是真话。

- 当前的话与前面的某些真的话冲突，就是假话
- 当前的话中 X 或 Y 比 N 大，就是假话
- 当前的话表示 X 吃 X，就是假话

你的任务是根据给定的 N 和 K 句话，输出假话的总数。



2024 食物链



输入 #1

```
100 7
1 101 1
2 1 2
2 2 3
2 3 3
1 1 3
2 3 1
1 5 5
```

输出 #1

```
3
```

并查集能维护连通性、传递性，通俗地说，亲戚的亲戚是亲戚。然而当我们需要维护一些对立关系，比如 敌人的敌人是朋友 时，正常的并查集就很难满足我们的需求。

这时，种类并查集就诞生了。

常见的做法是将原并查集扩大一倍规模，并划分为两个种类。详见洛谷题解。

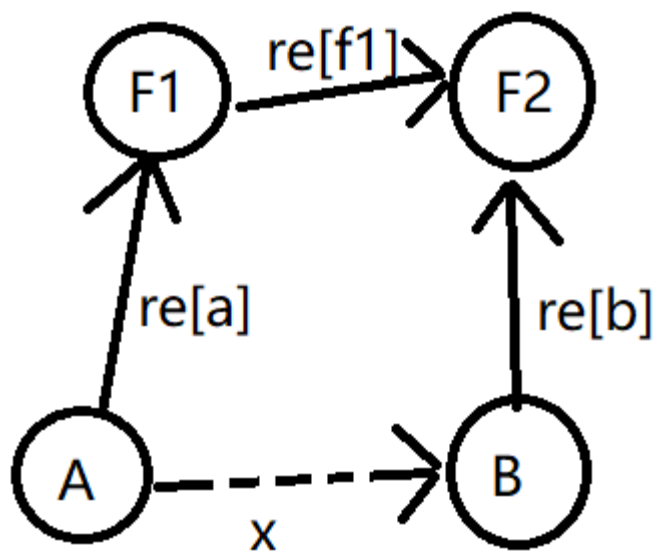
今天我们考虑利用权值来处理种类并查集。

$$1 \leq N \leq 5 * 10^4$$

$$1 \leq K \leq 10^5$$



2024 食物链



由题可知，两个物种之间的关系就三种。1：同类，2：食物，3：天敌。用 $re[i]$ 表示 i 与父亲 $fa[i]$ 的关系，所以权值分为三种：同类 $re[i]=0$ ；捕食关系 $re[i]=1$ ；天敌关系 $re[i]=2$ ；

初始化为0，即自己与自己为同种动物。

那么第一个问题，权值可否转移？

同类的同类是同类， re 相加=0；

食物的食物是天敌， re 相加=2；

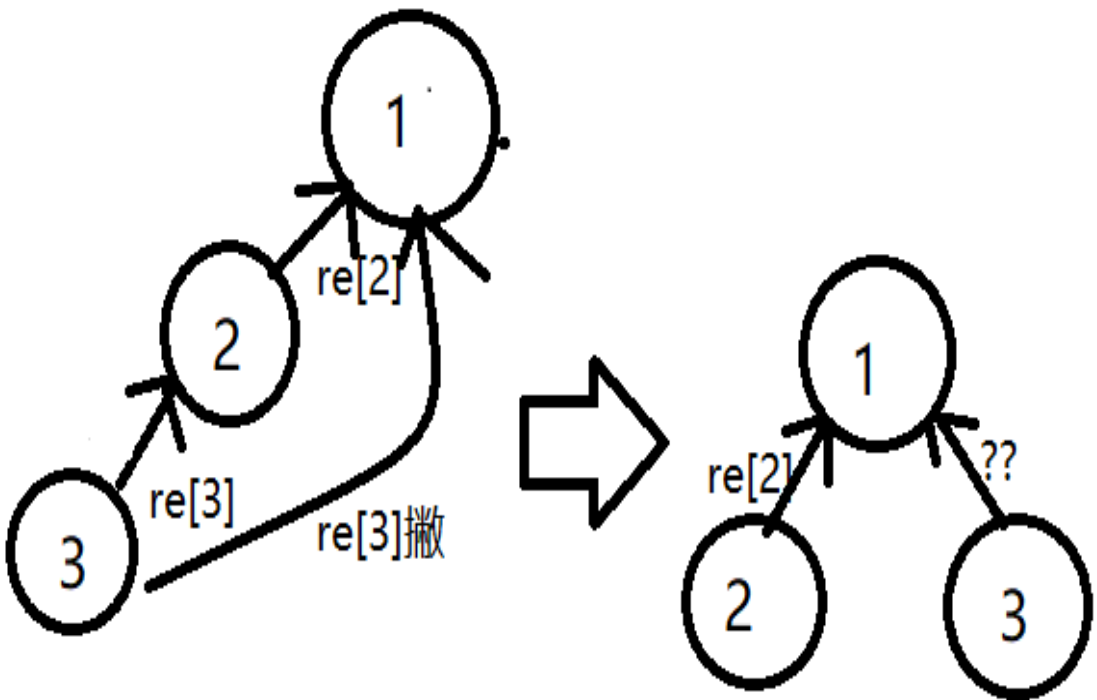
天敌的天敌是食物， re 相加=4；对3取余 $re=1$ ；

关系在转移时要对3取余。

那么第二个问题，在查找和合并时如何转移权值？



2024 食物链



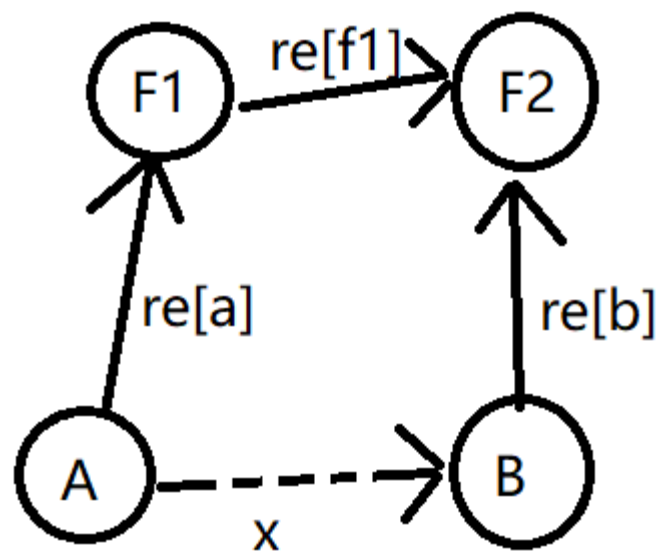
查找（路径压缩）：路径压缩就是在搜索的时候找到最远的祖先，然后将父亲节点赋值，对于权值而言，就是找出权值与最远祖先之前所有边权传递的过程，找出节点与父亲节点的关系，依次传递即可。

路径压缩后 $re[3]=?$

新的 $re[3]=(re[3]+re[2])\%3$;
不要忘记取余。



2024 食物链



1.合并：并查集合并的本质就是一棵树认另一棵树做父亲，把树根相连即可，但是能否也把权值直接赋值呢（比如1操作就直接赋值为1）？当然不行，因为给你的a,b是树下节点，还有考虑各自与树根的关系。也就是说，推出A,B各自与根的关系，就可以实现树根权值的连接了。

由图得， $re[f1]=x+re[b]-re[a]$

结果有可能 <0

$$re[f1]=(x+re[b]-re[a]+3)\%3$$



练习



- 练习题目：
 - p1551 亲戚
 - p1525 关押罪犯
 - P2024 食物链
 - P1111 修复公路
 - P1682 过家家
 - p1197星球大战
 - P2342 叠积木
-